Install packages in a virtual environment using pip and venv

This guide discusses how to create and activate a virtual environment using the standard library's virtual environment tool venv and install packages. The guide covers how to:

- Create and activate a virtual environment
- Prepare pip
- Install packages into a virtual environment using the `pip` command
- Use and create a requirements file

> **Note**
>
> This guide applies to supported versions of Python, currently 3.8 and higher.

> **Note**
>
> This guide uses the term **package** to refer to a Distribution Package, which commonly is installed from an external host. This differs from the term Import Package which refers to import modules in your Python source code.

> **Important**
>
> This guide has the prerequisite that you are using an official Python version obtained from <https://www.python.org/downloads/>. If you are using your operating system's package manager to install Python, please ensure that Python is installed before proceeding with these steps.

Create and Use Virtual Environments

Create a new virtual environment

venv (for Python 3) allows you to manage separate package installations for different projects. It creates a "virtual" isolated Python installation. When you switch projects, you can create a new virtual environment which is isolated from other virtual environments. You benefit from the virtual environment since packages can be installed confidently and will not interfere with another project's environment.

> **Tip**

Skip to content en latest

MongoDB 8.0 on Atlas: deploy worldwide with new regions on AWS, Azure & Google Cloud. Try free!

Ads by EthicalAds

Close Ad

To create a virtual environment, go to your project's directory and run the following command. This will create a new virtual environment in a local folder named `.venv`:

Unix/macOS **Windows**

```
python3 -m venv .venv
```

The second argument is the location to create the virtual environment. Generally, you can just create this in your project and call it `.venv`.

`venv` will create a virtual Python installation in the `.venv` folder.

> **Note**
>
> You should exclude your virtual environment directory from your version control system using `.gitignore` or similar.

Activate a virtual environment

Before you can start installing or using packages in your virtual environment you'll need to `activate` it. Activating a virtual environment will put the virtual environment-specific `python` and `pip` executables into your shell's `PATH`.

Unix/macOS **Windows**

```
source .venv/bin/activate
```

To confirm the virtual environment is activated, check the location of your Python interpreter:

Unix/macOS **Windows**

```
which python
```

While the virtual environment is active, the above command will output a filepath that includes the `.venv` directory, by ending with the following:

Unix/macOS **Windows**

```
.venv/bin/python
```

While a virtual environment is activated, pip will install packages into that specific environment. This enables you to import and use packages in your Python application.

Skip to content a virtual environment en latest

MongoDB 8.0 on Atlas: deploy worldwide with new regions on AWS, Azure & Google Cloud. Try free!

Ads by EthicalAds

Close Ad

```
deactivate
```

> **Note**
>
> Closing your shell will deactivate the virtual environment. If you open a new shell window and want to use the virtual environment, reactivate it.

Reactivate a virtual environment

If you want to reactivate an existing virtual environment, follow the same instructions about activating a virtual environment. There's no need to create a new virtual environment.

Prepare pip

pip is the reference Python package manager. It's used to install and update packages into a virtual environment.

Unix/macOS **Windows**

The Python installers for macOS include pip. On Linux, you may have to install an additional package such as `python3-pip`. You can make sure that pip is up-to-date by running:

```
python3 -m pip install --upgrade pip
python3 -m pip --version
```

Afterwards, you should have the latest version of pip installed in your user site:

```
pip 23.3.1 from .../.venv/lib/python3.9/site-packages (python 3.9)
```

Install packages using pip

When your virtual environment is activated, you can install packages. Use the `pip install` command to install packages.

Install a package

For example, let's install the Requests library from the Python Package Index (PyPI):

Unix/macOS **Windows**

Skip to content en latest

MongoDB 8.0 on Atlas: deploy worldwide with new regions on AWS, Azure & Google Cloud. Try free!

Ads by EthicalAds

Close Ad

pip should download requests and all of its dependencies and install them:

```
Collecting requests
  Using cached requests-2.18.4-py2.py3-none-any.whl
Collecting chardet<3.1.0,>=3.0.2 (from requests)
  Using cached chardet-3.0.4-py2.py3-none-any.whl
Collecting urllib3<1.23,>=1.21.1 (from requests)
  Using cached urllib3-1.22-py2.py3-none-any.whl
Collecting certifi>=2017.4.17 (from requests)
  Using cached certifi-2017.7.27.1-py2.py3-none-any.whl
Collecting idna<2.7,>=2.5 (from requests)
  Using cached idna-2.6-py2.py3-none-any.whl
Installing collected packages: chardet, urllib3, certifi, idna, requests
Successfully installed certifi-2017.7.27.1 chardet-3.0.4 idna-2.6 requests-2.18.4 urllib3-1.22
```

Install a specific package version

pip allows you to specify which version of a package to install using version specifiers. For example, to install a specific version of `requests`:

Unix/macOS **Windows**

```
python3 -m pip install 'requests==2.18.4'
```

To install the latest `2.x` release of requests:

Unix/macOS **Windows**

```
python3 -m pip install 'requests>=2.0.0,<3.0.0'
```

To install pre-release versions of packages, use the `--pre` flag:

Unix/macOS **Windows**

```
python3 -m pip install --pre requests
```

Install extras

Some packages have optional extras. You can tell pip to install these by specifying the extra in brackets:

Windows

Skip to content en latest

MongoDB 8.0 on Atlas: deploy worldwide with new regions on AWS, Azure & Google Cloud. Try free!

Ads by EthicalAds

Close Ad

Install a package from source

pip can install a package directly from its source code. For example, to install the source code in the `google-auth` directory:

Unix/macOS **Windows**

```
cd google-auth
python3 -m pip install .
```

Additionally, pip can install packages from source in development mode, meaning that changes to the source directory will immediately affect the installed package without needing to re-install:

Unix/macOS **Windows**

```
python3 -m pip install --editable .
```

Install from version control systems

pip can install packages directly from their version control system. For example, you can install directly from a git repository:

```
google-auth @ git+https://github.com/GoogleCloudPlatform/google-auth-library-python.git
```

For more information on supported version control systems and syntax, see pip's documentation on VCS Support.

Install from local archives

If you have a local copy of a Distribution Package's archive (a zip, wheel, or tar file) you can install it directly with pip:

Unix/macOS **Windows**

```
python3 -m pip install requests-2.18.4.tar.gz
```

If you have a directory containing archives of multiple packages, you can tell pip to look for packages there and not to use the Python Package Index (PyPI) at all:

Unix/macOS **Windows**

```
nstall --no-index --find-links=/local/dir/ requests
```

Skip to content en latest

MongoDB 8.0 on Atlas: deploy worldwide with new regions on AWS, Azure & Google Cloud. Try free!

Ads by EthicalAds

Close Ad

This is useful if you are installing packages on a system with limited connectivity or if you want to strictly control the origin of distribution packages.

Install from other package indexes

If you want to download packages from a different index than the Python Package Index (PyPI), you can use the `--index-url` flag:

Unix/macOS **Windows**

```
python3 -m pip install --index-url http://index.example.com/simple/ SomeProject
```

If you want to allow packages from both the Python Package Index (PyPI) and a separate index, you can use the `--extra-index-url` flag instead:

Unix/macOS **Windows**

```
python3 -m pip install --extra-index-url http://index.example.com/simple/ SomeProject
```

Upgrading packages

pip can upgrade packages in-place using the `--upgrade` flag. For example, to install the latest version of `requests` and all of its dependencies:

Unix/macOS **Windows**

```
python3 -m pip install --upgrade requests
```

Using a requirements file

Instead of installing packages individually, pip allows you to declare all dependencies in a Requirements File. For example you could create a `requirements.txt` file containing:

```
requests==2.18.4
google-auth==1.1.0
```

And tell pip to install all of the packages in this file using the `-r` flag:

Skip to content
en latest
MongoDB 8.0 on Atlas: deploy worldwide with new regions on AWS, Azure & Google Cloud. Try free!
Ads by EthicalAds
Close Ad

Freezing dependencies

Pip can export a list of all installed packages and their versions using the `freeze` command:

Unix/macOS **Windows**

```
python3 -m pip freeze
```

Which will output a list of package specifiers such as:

```
cachetools==2.0.1
certifi==2017.7.27.1
chardet==3.0.4
google-auth==1.1.1
idna==2.6
pyasn1==0.3.6
pyasn1-modules==0.1.4
requests==2.18.4
rsa==3.4.2
six==1.11.0
urllib3==1.22
```

The `pip freeze` command is useful for creating Requirements Files that can re-create the exact versions of all packages installed in an environment.

Copyright © 2013–2020, PyPA
Made with Sphinx and @pradyunsg's Furo
Last updated on Jan 20, 2026

en latest

MongoDB 8.0 on Atlas: deploy worldwide with new regions on AWS, Azure & Google Cloud. Try free!

Ads by EthicalAds

Close Ad